

MICROCONTROLLER- self-contained desktop

A microcontroller (MCU) is a small computer on a single integrated circuit that is designed to control specific tasks within electronic systems.

It combines the functions of a central processing unit (CPU), memory, and input/output interfaces, all on a single chip.

A typical microcontroller consists of a processor core, volatile and non-volatile memory, input/output peripherals, and various communication interfaces.

The processor core is responsible for executing instructions and controlling the other components of the microcontroller.

The memory is used to store data and program code, while the input/output peripherals are used to interact with the external environment.

The microcontroller chip is a high-speed device, yet it is slow when compared to a computer. Microcontroller consist of RAM, ROM , Timer , I/O Ports.ROM is used for program storage and RAM is used for data storage.



Microcontroller

The quartz oscillator (for clock generation) is enabled and through control logic register once the supply is powered on. Parasite capacitors will be recharged for a few seconds while the early preparation is taking place. Once the voltage level reaches its maximum value and the oscillator's frequency stabilizes, the operation of writing bits through special function registers becomes stable. Everything is controlled by the oscillator's CLK, and the whole electronics will begin to function.

Used in sensor, actuators (converts electrical signals into **physical movement**), display of washing machine

Types of Microcontrollers:

8-bit Microcontrollers: basic type of microcontrollers, typically used in simple applications

16-bit Microcontrollers: advanced than 8-bit microcontrollers and can perform more complex tasks.

32-bit Microcontrollers: most powerful and feature-rich microcontrollers, capable of handling large amounts of data and performing high-speed processing

MICROPROCESSOR- digital device

A Microprocessor is an important part of a computer architecture. It is a programmable device that takes in input performs some arithmetic and logical operations over it and produces the desired output. It can fetch instructions from memory, decode and execute them.

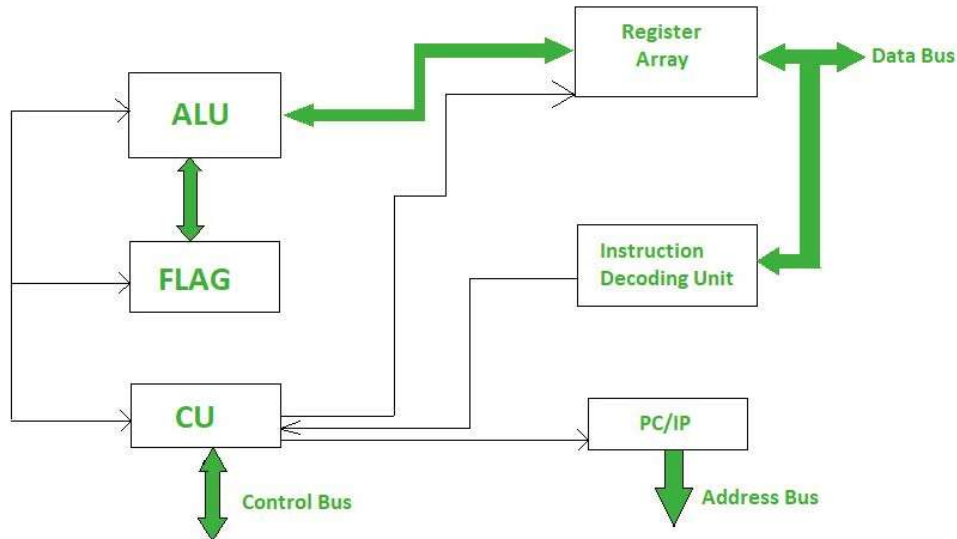
takes a bunch of instructions in machine language and executes them.

telling the processor what it must do.

It performs some basic operations like addition, subtraction, multiplication, division, and some logical operations using its Arithmetic and Logical Unit (ALU)

Data in microprocessors can move from one location to another

It has a Program Countert (PC) register that stores the address of the next instruction based on the value of the PC.(It is a 16-bit register and is also called instruction counter, instruction pointer, and instruction address register (IAR). PC (program counter) is a digital counter that is needed to execute tasks quickly and track the current execution point.)



Flags indicate results of ALU operations (e.g., whether the result is zero or if there was a carry). Register array-stores temporary data and instructions (internal memory). Decoding unit- decodes the received instruction and tell the processor what to do next. PC/IP (program counter/instruction pointer)- holds address of next instruction to be executed. Increments after each fetch.

Data Bus (external memory access) – transfers data between registers, memory and ALU. Address Bus- carries memory address from PC/IP to access data. Control Bus- carries control signals from control unit to other components.

TYPES-

CISC – Complex instruction set computer -single instruction executing multiple low-level operations.

RISC- “Reduced”-Instructions get completed in one clock cycle this is because of the optimization of instructions and pipelining (a technique that allows for simultaneous execution of parts, or stages, of instructions more efficiently process instructions).

Basic Terms used in Microprocessor

Given below are some basic terms used in the microprocessors

- **Instruction Set:** Instruction set is a group of instructions. It is the Interface between hardware and software.
- **Bus:** used to carry data (data bus), address (address bus) and control signal(control bus) within a microprocessor.
- **IPC (Instructions Per Cycle):** the number of instructions that a CPU can execute in a Single clock cycle.

- Clock Speed: number of operations a processor can perform per second. It is Measured in megahertz (MHz) or gigahertz (GHz).
- Bandwidth: number of bits processed during a single instruction.
- Word Length: Shows the number of bits a processor can handle at a time.
- Data Types: It Supports various formats such as binary, ASCII, Signed and Unsigned Bits.

DIFFERENCE BETWEEN MICROCONTROLLER AND MICROPROCESSOR

MICROCONTROLLER (general purpose application)	MICROPROCESSOR (Application specific system)
Used to perform specific tasks. (Camera, washing machine, microwave)	Used where the task is user-defined and not pre-defined. (Gaming, photo editing)
Used in embedded systems where real-time control, automation is needed.	Used where intensive processing is required.
Contains CPU, I/O, memory units – all integrated internally	Contains only a CPU, memory, timers and I/O units connected externally.
Overall size is comparatively smaller.	Overall size of system is larger
CLK speed: MHz RAM: 2KB to 256KB (memory is less as they are specified for a task) Flash memory: 32KB to 2MB	Operated at much higher speed: CLK speed: GHz RAM: 512MB to 32GB ROM (Hard disk): 128GB to 2TB

Peripheral interfaces: USB, High speed Ethernet, UART.	Peripheral interfaces: 12C, SPI, UART
8 bits, 16 bits and 32 bits	32 bits and 64 bits
Low power consumption, lower cost and task specific.	High power consumption, higher cost and multitasking
Microcontrollers have no advantage of designing RAM, ROM, I/O port.	Microcontrollers have no advantage of designing RAM, ROM, I/O port.
Uses Harvard architecture.	Uses Neumann architecture.
Have a zero-status flag, less no of register.	Doesn't have a zero-status flag, have more no. of registers.

UART (Universal Asynchronous Receiver-Transmitter)- two-wire communication ((TX, RX): Used in GPS modules, Bluetooth modules.**SPI (Serial Peripheral Interface)**- Faster than UART, supports multiple devices (Master-Slave): SD cards, OLED displays, Flash memory, sensors.**I2C (Inter-Integrated Circuit)**- only two wires (SDA, SCL)- multiple devices- temperature sensors (DS3231), LCDs, accelerometers (MPU6050).

SoCs (System on chip) - An SoC (System on Chip) integrates a microprocessor, microcontroller, memory, I/O peripherals, and sometimes even wireless communication onto a single chip

Harvard architecture: Separate memory and buses, can fetch instructions simultaneously. **Von Neumann architecture:** Instructions and data share the same memory and bus.

MICROCONTROLLER: ADVANTAGES AND DISADVANTAGES

Advantages:

- 1) **Time required:** Low time required.
- 2) **Multitasking:** At an equivalent time, many tasks are often performed.
- 3) **Low power consumption:** extend the battery life of electronic devices.
- 4) **Small size:** Microcontrollers are small and compact.
- 5) **Cost-effective:** Microcontrollers are generally less expensive.

Disadvantages:

- 1) Microcontrollers are composed of complementary metal-oxide-semiconductor (CMOS) and can be damaged by a static charge.
- 2) Memory capacity is much lower than that of a microprocessor, hence it cannot be utilized for high-level computing tasks.
- 3) Number of **executions** is limited. Limited **memory** capacity. may not have the ability to **connect to external** networks or devices, which can limit their functionality. may have limited **software support**.

MICROPROCESSOR: ADVANTAGES AND DISADVANTAGES

- 1) have an extremely high processing or computational speed.
- 2) capabilities of data-driven decision-making.
- 3) Since the devices are programmable, there is flexibility to alter the system by changing the software alone.
- 4) They can be easily integrated with a wide range of external peripherals like sensors, displays
- 5) Software and firmware can be updated to enhance functionality without changing the hardware

- 1) limitation on the size of data . Microprocessors don't have built-in RAM or ROM, so external memory is required.
- 2) Microprocessors are not ideal for real-time applications requiring immediate response, as they typically operate in a sequential manner.
- 3) Basic microprocessors generally lack built-in security features, making them vulnerable to hacking
- 4) over heating physically. High-performance microprocessors generate more heat and may require additional cooling solutions.

ARDUINO UNO (Based on ATmega328P)

Arduino Uno is based on both the ATmega328P and the ATmega16U2 processors(BOTH ARE MICROCONTROLLER). Has a built in USART.

ATmega328P: main microcontroller or the "brain" of the Arduino Uno. executes the code you write, controls the I/O pins, interacting with peripherals.

Specifications:

- 32 KB Flash Memory (for storing the code).
- 2 KB SRAM (for variables).
- 1 KB EEPROM (for permanent storage).
- 8-bit processor running at 16 MHz. AVR CPU

ATmega16U2: USB-to-Serial converter. The program written in Arduino IDE is sent as serial data through USB. ATmega16U2 converts this USB data into serial (RX/TX) signals for the ATmega328P microcontroller to receive.

DIFFERENT TYPE OF MEMORIES IN ARDUINO

1. Flash Memory: where your compiled code is saved

- Type: Non-volatile (retains data even when power is off).
- Purpose: Stores the program code (your Arduino sketch).
- Size in Arduino Uno: 32 KB

2. SRAM (Static Random-Access Memory):

- Type: Volatile (data is lost when power is off).
- Purpose: Stores variables and data used during program execution.
- Size in Arduino Uno: 2 KB

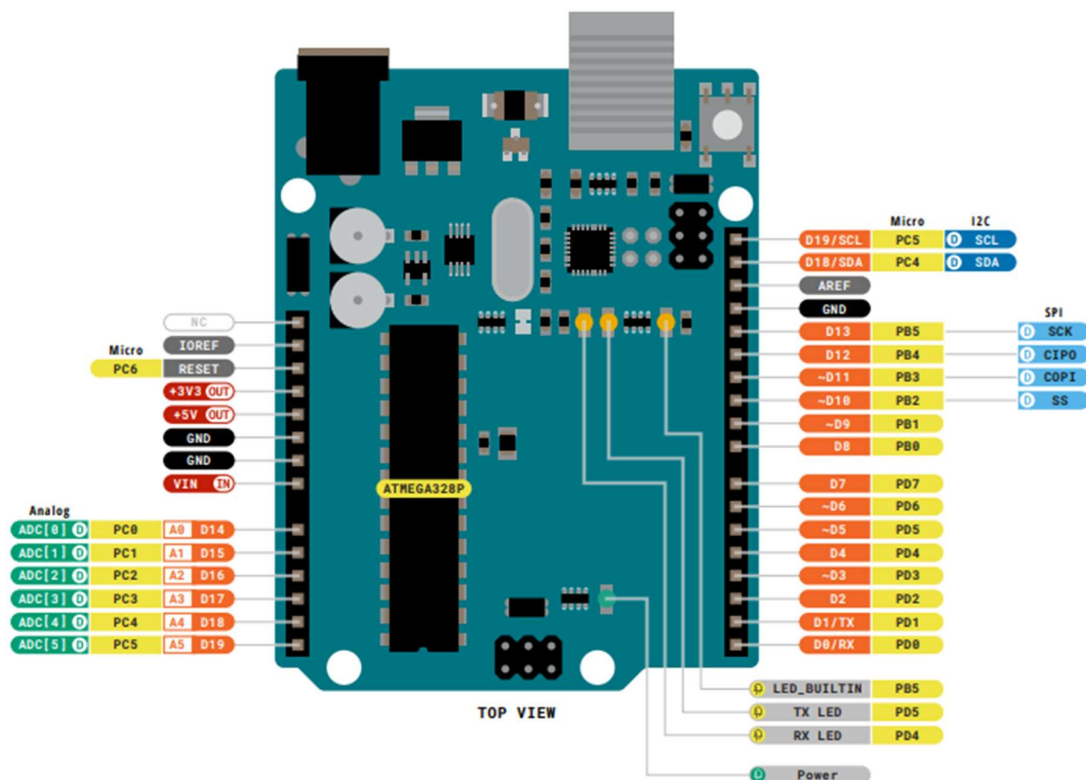
3. EEPROM (Electrically Erasable Programmable Read-Only Memory):

- Type: Non-volatile.
- Purpose: Stores persistent data (e.g., settings, calibration values) that must be preserved even after power is turned off.
- Size in Arduino Uno: 1 KB

4. ROM (Read-Only Memory):

- In the context of the Arduino Uno, **ROM** is not present as a separate module.
- The **Flash Memory** acts as the ROM because it stores the program code, which is "read-only" during execution.

Pins and components:



Power Pins:

1. **VIN:** Input voltage to the Arduino board (7-12V).
2. **GND:** Ground pins.
3. **3.3V:** Output of 3.3V (max 50 mA).
4. **5V:** Output of 5V, used to power external components.

Analog Input Pins: 6

- **A0 to A5:** These pins are used to read analog input signals (e.g., from sensors) and convert them to digital values using the internal ADC (Analog to Digital Converter). ATmega328P is digital by design. Also for serial communication.

Digital Input/Output Pins: 14

- **D0 to D13:** General-purpose digital input/output pins.
 - **D0 (RX) and D1 (TX):** Used for serial communication.
 - **D2 to D13:** Can be used as digital I/O. Some of these have special functions:
 - **D3, D5, D6, D9, D10, D11:** PWM (Pulse Width Modulation) 6 outputs.
 - **D13:** Connected to the built-in LED.

Communication Pins:

1. UART (Serial Communication):

- **D0 (RX):** Receive data.
- **D1 (TX):** Transmit data.

2. I2C Communication:

- **SDA (A4) and SCL (A5):** Used for I2C communication.

SDA and SCL Pins:

- **SDA (A4): Serial Data Line** – Transfers data between devices.
- **SCL (A5): Serial Clock Line** – Provides the clock signal to synchronize communication.

3. SPI Communication:

- **SCK (D13), MISO (D12), MOSI (D11), SS (D10):** Used for SPI communication with other devices.

Special Pins:

1. **RESET:** Resets the microcontroller.
2. **AREF:** Analog Reference for analog inputs.
3. **IOREF:** Provides the voltage reference (5V or 3.3V) to the shields connected.

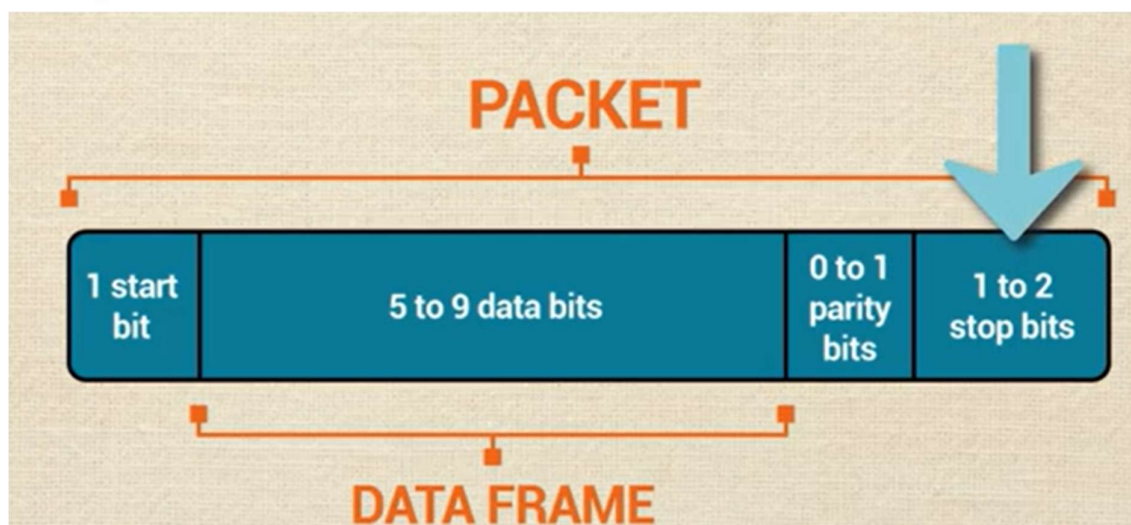
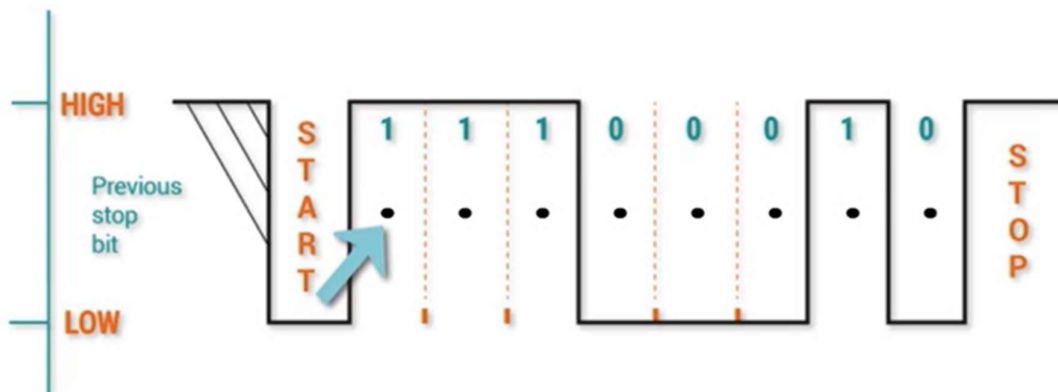
On-board Components:

1. **Microcontroller: ATmega328P** – The main brain of the board.
2. **USB Connector:** Used to program and power the Arduino.
3. **Power Jack:** Alternative power input (7-12V).
4. **Crystal Oscillator:** Ensures accurate timing for the microcontroller (16 MHz).
5. **Voltage Regulator:** Regulates the input voltage to 5V or 3.3V as needed.
6. **Built-in LED:** Connected to digital pin 13 for testing purposes.

TERMS EXPLAINED:

Serial communication: is a method of sending and receiving information one bit at a time. Over USB (Universal serial bus) it is a serial communication protocol. Other ways: I2C and TTL serial using USART.

USART: uses the RX TX pins and send bits of data serially to a different microcontroller on the board. This communication is **TTL serial** that is transistor-transistor logic .



Baud rate is how many times per second the signal could change. If it is 9600 that means the signal could change every $1/9600$ second .

I2C COMMUNICATION: serial communication protocol. two-wire communication protocol that allows multiple devices (sensors, displays, etc.) to communicate with the Arduino using a master-slave architecture(used in read heavy systems – where read operations are more common than write – slave is copy of main database – read requests are taken by slave and write request by master—having multiple slaves if the master server goes down, one slave can be promoted to master)

HOW I2C WORKS?

One device acts as the **Master** (usually the Arduino). Other devices are **Slaves** (e.g., sensors, displays). **Master sends the clock signal** on SCL. **Data is sent/received** on the SDA line.

SPI COMMUNICATION: SPI (Serial Peripheral Interface) is a high-speed serial synchronous communication protocol. uses **four lines** for data transfer between a **Master** (e.g., Arduino) and one or more **Slaves** (e.g., sensors, SD cards).

PINS:

- **SCK (D13) – Serial Clock:** Clock signal generated by the Master to synchronize data transfer.
- **MISO (D12) – Master In Slave Out:** Data line for receiving data from the Slave to the Master.
- **MOSI (D11) – Master Out Slave In:** Data line for sending data from the Master to the Slave.
- **SS (D10) – Slave Select:** Used to select the specific Slave device for communication.

ARDUINO NANO

Based on ATmega328P – microcontroller and FT232RL or CH340 chip (USB-to-Serial interface ICs)

FT232RL, **CH340**, and **ATmega16U2** all provide **USB-to-Serial communication** but differ in complexity

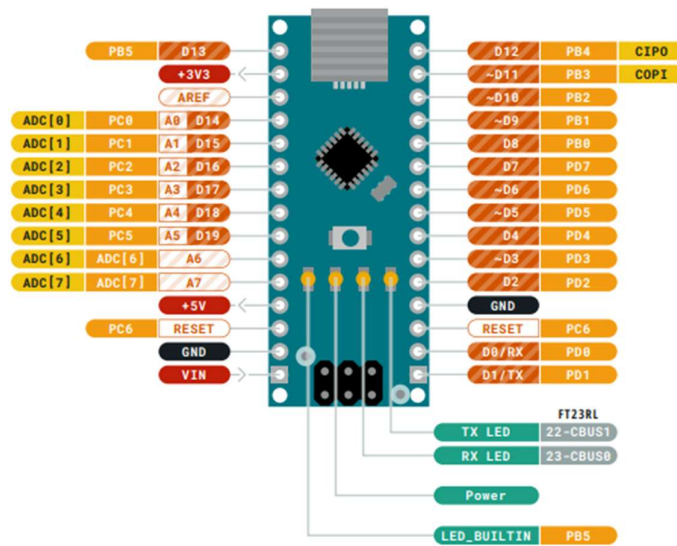
FT232RL and **CH340:**

- These are **dedicated USB-to-Serial interface ICs**.
- Their **only job** is to **convert USB signals** to **UART (TX/RX)** serial communication.
- They **cannot be reprogrammed** for other tasks.

ATmega16U2:

- It is a **microcontroller** that **acts as a USB-to-Serial converter** on the **Arduino Uno**.
- It is **programmable**, meaning it **can do more** than just USB-to-Serial conversion

SPECIFICATION: same as UNO .



Power Pins:

- **VIN:** Input voltage to the Arduino when using an external power source (7-12V).
- **+5V:** Regulated 5V output from the onboard voltage regulator.
- **+3.3V:** 3.3V output from the onboard regulator.
- **GND:** Ground pins.
- **RESET:** Reset the microcontroller.

Analog Input Pins (A0 to A7):

- Labeled as ADC[0] to ADC[7].
- Can read analog signals (0-5V) and convert them to a 10-bit digital value.

Digital Input/Output Pins (D0 to D13):

- Can be used as input or output using `pinMode()`, `digitalRead()`, and `digitalWrite()` functions.
- **D0 (RX) and D1 (TX):** Used for serial communication.

- **D3, D5, D6, D9, D10, D11:** PWM output pins.

SPI Pins:

- **D11 (MOSI), D12 (MISO), D13 (SCK)** for SPI communication.

I2C Pins:

- **A4 (SDA) and A5 (SCL)** for I2C communication.

Built-in LED:

- **LED_BUILTIN (D13):** Connected to a built-in LED for testing and debugging.
- • **14 Purely Digital Pins:** D0 to D13
- • **6 Dual-Function Pins:** A0 to A5 (can be used as either digital I/O or analog input)
- • **2 Purely Analog Pins:** A6 and A7 (analog input only, no digital I/O capability)

DIFFERENCE BETWEEN ARDUINO UNO AND ARDUINO NANO:

Size and Form Factor:

- **Uno:** Larger, with a standard rectangular shape (68.6 mm × 53.4 mm).
- **Nano:** Smaller and more compact (45 mm × 18 mm), suitable for breadboards and compact projects.

Pin Count:

- **Uno:**
 - 14 Digital I/O pins (D0 to D13)
 - 6 Analog Input pins (A0 to A5)
- **Nano:**
 - 14 Digital I/O pins (D0 to D13)
 - 8 Analog Input pins (A0 to A7, with A6 and A7 being analog-only)

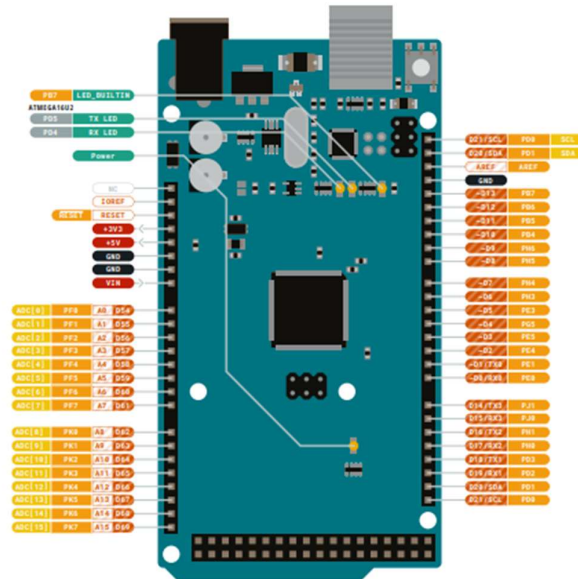
Programming:

- **Uno:** Uses the **ATmega16U2** chip for USB-to-serial communication.
- **Nano:** Uses the **FT232** or **CH340** (on newer clones) for USB-to-serial communication.

ARDUINO MEGA(Based on ATmega2560)

SPECIFICATION:

ARDUINO



Digital I/O Pins:

- 54 Digital I/O Pins (D0 to D53)
 - 15 of these can provide PWM output (Pins: 2 to 13, 44 to 46).

Analog Input Pins:

- 16 Analog Inputs (A0 to A15)
 - Each analog input provides 10-bit resolution (0-1023).

Communication Pins:

- 4 Serial Ports (UARTs):
 - Serial 0: RX0 (D0), TX0 (D1)
 - Serial 1: RX1 (D19), TX1 (D18)
 - Serial 2: RX2 (D17), TX2 (D16)
 - Serial 3: RX3 (D15), TX3 (D14)

SPI Pins:

- MOSI (D51), MISO (D50), SCK (D52), SS (D53)

I2C Pins:

- **SDA (D20), SCL (D21)**

PWM Pins:

- **15 PWM Output Pins**
 - (2 to 13, 44 to 46)

Power Pins:

- **VIN:** External power input (7-12V).
- **+5V:** Regulated 5V output.
- **+3.3V:** 3.3V output (maximum 50 mA).
- **GND:** Ground pins.
- **RESET:** Reset the microcontroller.

DIFFERENCE IN BETWEEN ARDUINO NANO/UNO AND ARDUINO MEGA

Memory:

- **ATmega2560:**
 - **Flash Memory:** 256 KB
 - **SRAM:** 8 KB
 - **EEPROM:** 4 KB
- **ATmega328P:**
 - **Flash Memory:** 32 KB (0.5 KB used by bootloader)
 - **SRAM:** 2 KB
 - **EEPROM:** 1KB

PWM Channels:

- **ATmega2560:** Four 8-bit PWM Channels
- **ATmega328P:** Six PWM Channel

USART:

- **ATmega2560:** Four Programmable Serial USARTs
- **ATmega328P:** One USART with fractional baud rate generator and start-of-frame detection

Timers and Counters:

ATmega2560:

- Multiple 8-bit and 16-bit timers

ATmega328P:

- 2x 8-bit Timer/Counters
- 1x 16-bit Timer/Counter

COMMUNICATION PROTOCOLS ON ARDUINO

I2C (INTER INTEGRATED CIRCUIT)

It is incorporated into devices for serial communication. It is also known as Two Wired Interface (TWI) for short-distance communication.

Working of I2C Communication Protocol

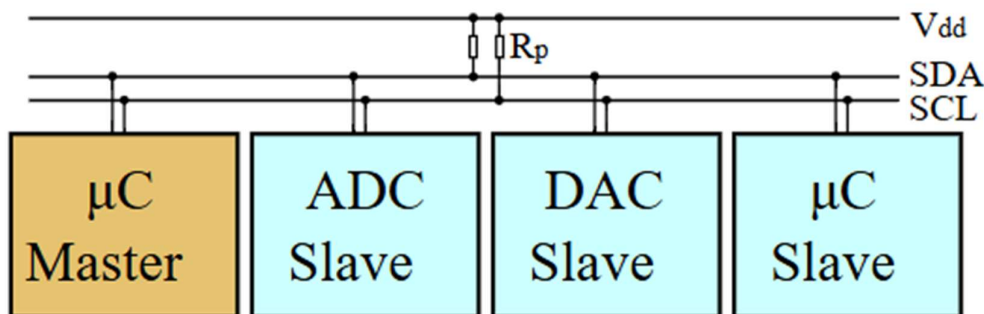
It uses only 2 bi-directional open-drain lines for data communication called SDA and SCL.

Serial Data (SDA) : Transfer of data takes place through this pin.

Serial Clock (SCL) : It carries the clock signal.

I2C can operate in two modes: MASTER mode and SLAVE mode

MASTER AND SLAVE MODES:



MASTER → SLAVE

- 1) Only the master can start and stop a communication sequence on the I²C bus.
- 2) Broadcast slave address (7 bits) + the R/W (8th) bit set to WRITE
- 3) Slave responds with **ACK (Acknowledgement)** if it recognizes the address or NACK (negative acknowledgement) if it wants to stop reading and master sends one byte . The step repeats zero or more times.

- 4) **Master:** Broadcast **STOP** (to end communication) or **RESTART** (to initiate another operation).

SLAVE → MASTER

Only difference is the bit in the second step would be set to **READ** which tells that the master is in the reading mode now.

Each data bit transferred on SDA line is synchronized by a high to the low pulse of each clock on the SCL line. Data on the **SDA line** is only considered valid and stable when the **SCL line** is **high**. The actual reading or sampling of the data bit happens when the SCL line transitions from **high to low**. data on the **SDA (Serial Data)** line can **only change** when the **SCL (Serial Clock)** line is **low**.

In I2C communication, the data line cannot change when the clock is high. Only when SCL is low. When SCL is high, the data on SDA is considered stable and is read by receiving device.

The 2 lines are open drain, hence a pull-up resistor is required so that the lines are high since the devices on the I2C bus are active low.

OPEN DRAIN AND PULL-UP RESISTOR: The device can only pull the line low but cannot drive it high. A **pull-up resistor** is required to keep the lines **high** when no device is pulling them low. This allows multiple devices to share on the same bus as any devices can pull the line low to communicate. I2C are active low, a low voltage represents a logic 0 and high voltage by logic 1 making the bus bidirectional allowing any device to communicate by pulling the line low.

The data is transmitted in the form of packets which comprises 9 bits:

1. **Start Condition:** 1 bit- master device sends a start condition by pulling the SDA line low while the SCL line is high
2. **Slave Address:** 8 bit- master sends the 7-bit address of the slave device it wants to communicate with, followed by a read/write bit- A high Read/Write bit indicates that the master is sending the data to the slave, whereas a low Read/Write bit indicates that the master is receiving data from the slave.
3. **Acknowledge:** 1 bit- The addressed slave device responds by pulling the SDA line low



When the transmission is complete, the master sends a stop condition by releasing the SDA line to high while the SCL line is high.

FEATURES OF I2C:

- 1) Half-Duplex: Bi-directional communication
- 2) Synchronous communication: data is sent in the form of blocks or frames. Both the sender and receiver are synchronized with a common clock signal. This means they operate at the same speed and know exactly when to send and receive data. Data is sent in a continuous stream with no time gap present between data.
- 3) Clock-stretching: **clock stretching** allows a **slave device** to **slow down** the communication if it's **not ready** to process more data. Slave holds the SCL line low which is usually kept high by the pull-up resistor. The master tries to raise the clock high but slave can hold the SCL low, it needs more time. The wires are AND WIRED means if any device pulls the SCL low it remains low regardless of other devices trying to raise it. It **prevents data loss or corruption** by ensuring the slave is always ready before the next bit is transferred.
- 4) Arbitration: process of **resolving conflicts** when **multiple masters** try to **control the bus simultaneously**. I2C supports a multi-master system, meaning more than one master can be connected to the same bus but only master can control the bus at a given time, prevent data collision.

IN ARDUINO:

SDA and SCL(A4 A5 – 20 21 for mega) pins are used for communication. To use I2C we need to include the wire library. Arduino has **built-in hardware support** for I²C communication. Some Arduino boards have **built-in pull-ups**, but if communication isn't stable, consider adding external resistors.

UART(Universal Asynchronous Receiver Transmitter)

U stands for Universal which means this protocol can be applied to any transmitter and receiver

A is for Asynchronous which means one cannot use clock signal for communication of data

R and **T** refers to Receiver and Transmitter

HOW IS SERIAL COMMUNICATION BETTER THAN PARALLEL COMMUNICATION and vice versa?

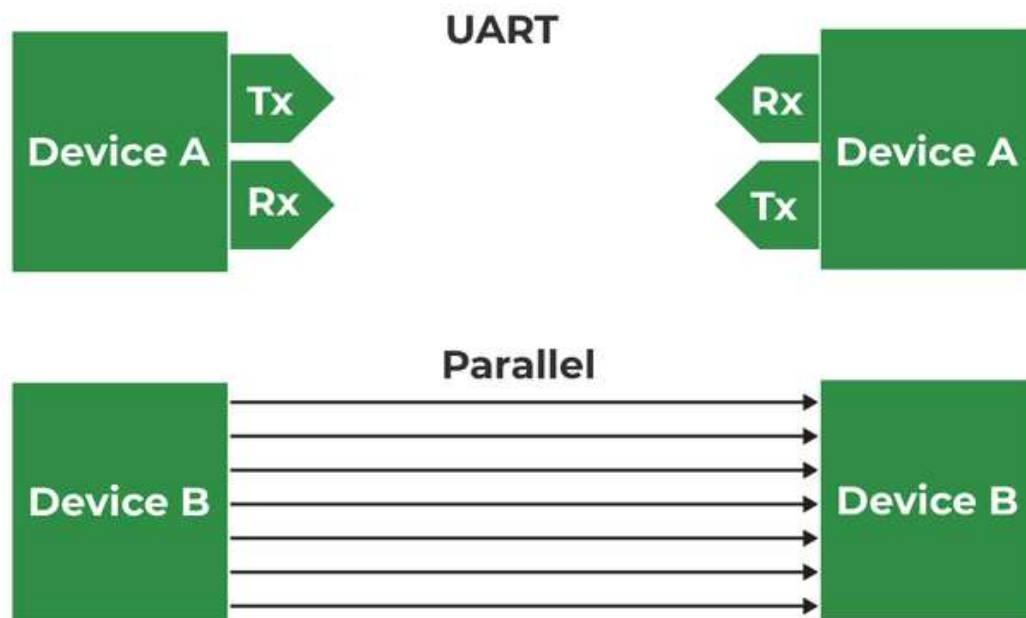
Fewer Wires Needed:

- **Serial Communication** uses **one wire** for data (plus ground).
- **Parallel Communication** needs **multiple wires** (e.g., 8 wires for 8 bits).

FOR PARALLEL:

Higher Data Throughput (Short Distances):

- **Parallel communication** can transfer **multiple bits simultaneously** (e.g., 8 or 16 bits at a time).
- This allows for **higher data throughput** compared to serial, but only over **short distances** where signal skew isn't an issue.



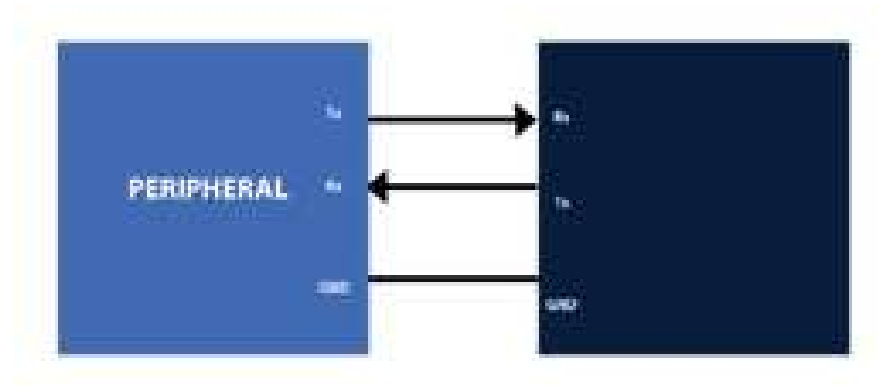
If DEVICE A wants to send data, then it will be sending data on the transmitter's pin and here receiver of this DEVICE B will receive it over and if DEVICE A wants to receive the data, then that is possible on the RX line that will be forwarded by TX of DEVICE B

UART must be configured before transmission. The Baud Rate for DEVICE A and DEVICE B must be same, otherwise both devices cannot understand at what speed and at what rate data is coming.

Configure the data length so here DEVICE A and DEVICE B both are configured at fixed data length if DEVICE A is transmitting data, then it is configured with fixed

data. Like if DEVICE A is configured with the eight-bit size of data then DEVICE B should also be configured at the same size of data which is eight bits.

In UART, possibilities to exchange data between the transmitter and receiver can be divided into three modes, namely simplex, half-duplex systems and full-duplex system



Simplex Systems: In this type of communication, the data bits are transmitted only from a single end (i.e., source device).

Half Duplex Systems: Data transmission can be achieved from either direction, but only one user can transmit data at a time. Since the hardware structure shares the same data bus and control buffer, users are allowed to either receive or transmit data at that instant.

Full Duplex Systems: Data transmission can happen in either direction, both the devices which are communicating through this system can receive and transmit data at the same time and they also have individual data buffers for Tx and Rx.

UART manages serial communication between devices by **converting data between serial and parallel forms**. In UART communication, data is transferred serially (bit by bit) but processed in parallel (as a group of bits, e.g., a byte) inside devices like microcontrollers or computers.

1. UART Transmitter Section:

1. Transmitter Holding Register:

- Purpose: Temporarily holds the data before transmission.
- **Function:** Acts as a buffer to avoid data loss.

2. Transmitter Shift Register:

- Purpose: Converts parallel data (e.g., 8-bit byte) into serial data.

- Function: Shifts bits one by one to the TX line.

3. Control Logic (Transmitter):

- Purpose: Manages the flow of data and error handling.
- Function:
 - Controls the sequence of data transfer between the holding register and shift register.
 - Ensures no blank or error data is sent.

2. UART Receiver Section:

1. Receiver Shift Register:

- Purpose: Converts serial data into parallel data.
- Function: Receives data bit by bit from the RX line.

2. Receiver Holding Register:

- Purpose: Temporarily stores the received data before processing.
- Function: Acts as a buffer to prevent data loss.

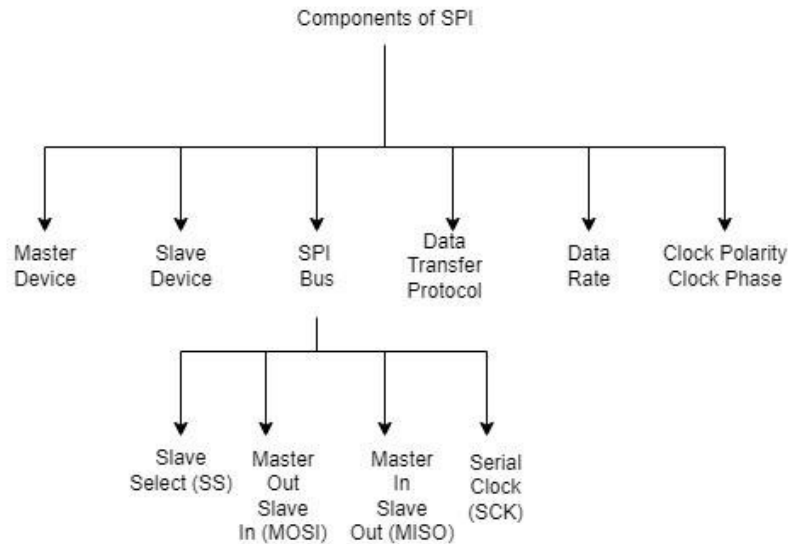
3. Control Logic (Receiver):

- Purpose: Manages read and write operations of received data.
- Function:
 - Coordinates data flow from the shift register to the holding register.
 - Checks for errors like framing errors (missing stop bit) or parity errors (incorrect parity bit).

SPI (Serial Peripheral Interface)

It is a protocol that is synchronous serial communication. It is used to communicate between the peripheral devices i.e. input and output devices and microcontrollers.

It facilitates the full-duplex, synchronous serial communication between one or more slave devices and a microcontroller.



Master Device: The master device is nothing, but it controls the process of transformation of data on the SPI bus.

Slave Device: Slave devices are peripheral devices that are connected to the SPI bus.

SPI Bus: SPI bus is a physical connection over the data transferring between the slave devices and the master.

- **Slave Select (SS):** In Slave Select, each slave device contains a dedicated SS pin. If the master will communicate with the specific slave. Multiple slave devices can be shared with the as same as MOSI, MISO, and SCK lines but it must have separated SS lines.
- **Master Out Slave In (MOSI):** In Master Out Slave In, MOSI can share the data or information from the master to other slave devices.
- **Master In Slave Out (MISO):** In Master In Slave Out, MISO can share the data or information from the slave device with the master.
- **Serial Clock (SCK):** In Serial Clock, this clock signal is used by the master and the slave devices for coordinating the data transfer timings

The data is transferred and received at the same time in full duplex mode.

Data Rate: The SPI bus can support the different data transferring rates depending upon the master capabilities of the slave devices and the transmission line's length.